

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
# Load final forecast
forecast = pd.read_csv("/kaggle/final_6_week_forecast.csv")

# Load inventory data
inventory = pd.read_csv("/kaggle/bm_inventory.csv")

# Load SKU master
skus = pd.read_csv("/kaggle/bm_skus.csv")

# Convert dates
inventory["snapshot_date"] = pd.to_datetime(inventory["snapshot_d

print("Data loaded successfully!")

print("\nForecast shape:", forecast.shape)
print("Inventory shape:", inventory.shape)
print("SKU master shape:", skus.shape)

print("\nForecast columns:")
print(forecast.columns.tolist())

print("\nInventory columns:")
print(inventory.columns.tolist())
```

Data loaded successfully!

Forecast shape: (1200, 3)
Inventory shape: (8735, 7)
SKU master shape: (200, 7)

Forecast columns:
['week', 'sku_id', 'forecast_demand']

Inventory columns:
['store_id', 'sku_id', 'stock_on_hand', 'reorder_point', 'safety_stock', 'las

```
print("Inventory snapshot dates:")
print(inventory["snapshot_date"].value_counts())

print("\nInventory summary:")
print(
    inventory[
        ["stock_on_hand", "reorder_point", "safety_stock"]
    ].describe()
```

```

    j.describe()
)

print("\nUnique stores:", inventory["store_id"].nunique())
print("Unique SKUs:", inventory["sku_id"].nunique())

print("\nMissing values:")
print(inventory.isna().sum())

```

Inventory snapshot dates:

snapshot_date

2025-10-31 8735

Name: count, dtype: int64

Inventory summary:

	stock_on_hand	reorder_point	safety_stock
count	8735.000000	8735.000000	8735.000000
mean	170.763938	67.755238	33.629193
std	77.875511	32.988546	16.499030
min	35.000000	10.000000	5.000000
25%	112.000000	43.000000	21.000000
50%	156.000000	62.000000	31.000000
75%	217.000000	87.000000	43.000000
max	449.000000	217.000000	108.000000

Unique stores: 50

Unique SKUs: 200

Missing values:

store_id	0
sku_id	0
stock_on_hand	0
reorder_point	0
safety_stock	0
last_restock_date	0
snapshot_date	0
dtype:	int64

```

# Aggregate inventory across all stores for each SKU

```

```

inventory_by_sku = (
    inventory
    .groupby("sku_id", as_index=False)
    .agg(
        stock_on_hand=("stock_on_hand", "sum"),
        reorder_point=("reorder_point", "sum"),
        safety_stock=("safety_stock", "sum")
    )
)

print("Inventory aggregated to SKU level!")

print("Shape:", inventory_by_sku.shape)
print("Number of SKUs:", inventory_by_sku["sku_id"].nunique())

print("\nFirst 10 SKUs:")
print(inventory_by_sku.head(10))

```

```
print("\nMissing values:")
print(inventory_by_sku.isna().sum())
```

Inventory aggregated to SKU level!

Shape: (200, 4)

Number of SKUs: 200

First 10 SKUs:

	sku_id	stock_on_hand	reorder_point	safety_stock
0	1001	5919	2417	1198
1	1002	7783	3071	1526
2	1003	7089	2770	1374
3	1004	9963	3980	1978
4	1005	7182	2808	1392
5	1006	8453	3306	1642
6	1007	5874	2373	1175
7	1008	7055	2781	1379
8	1009	4380	1789	884
9	1010	10543	4248	2113

Missing values:

```
sku_id      0
stock_on_hand 0
reorder_point 0
safety_stock 0
dtype: int64
```

```
# Calculate total forecast demand across the next 6 weeks for each SKU
forecast_by_sku = (
    forecast
    .groupby("sku_id", as_index=False)
    .agg(
        forecast_6_week_demand=("forecast_demand", "sum"),
        avg_weekly_forecast=("forecast_demand", "mean"),
        min_weekly_forecast=("forecast_demand", "min"),
        max_weekly_forecast=("forecast_demand", "max")
    )
)

print("6-week forecast aggregated successfully!")

print("Shape:", forecast_by_sku.shape)

print("\nFirst 10 SKUs:")
print(forecast_by_sku.head(10))

print("\nMissing values:")
print(forecast_by_sku.isna().sum())
```

6-week forecast aggregated successfully!

Shape: (200, 5)

First 10 SKUs:

	sku_id	forecast_6_week_demand	avg_weekly_forecast	min_weekly_forecast
0	1001	119.802438	19.967073	18.417987

1	1002	119.812083	19.968681	18.268207
2	1003	119.660684	19.943447	18.113649
3	1004	120.019339	20.003223	18.417987
4	1005	120.646742	20.107790	18.587716
5	1006	119.824481	19.970747	18.201344
6	1007	120.017444	20.002907	18.445166
7	1008	118.876042	19.812674	18.084599
8	1009	120.842209	20.140368	18.203883
9	1010	119.878503	19.979751	18.147021

```

max_weekly_forecast
0      22.860635
1      22.994554
2      22.983572
3      23.024297
4      22.894750
5      23.089139
6      22.934744
7      22.931453
8      24.944512
9      23.003006

```

```

Missing values:
sku_id      0
forecast_6_week_demand  0
avg_weekly_forecast     0
min_weekly_forecast     0
max_weekly_forecast     0
dtype: int64

```

```

# Combine 6-week forecast with current inventory

inventory_risk = inventory_by_sku.merge(
    forecast_by_sku,
    on="sku_id",
    how="left"
)

print("Forecast and inventory merged successfully!")

print("Shape:", inventory_risk.shape)

print("\nColumns:")
print(inventory_risk.columns.tolist())

print("\nFirst 10 rows:")
print(inventory_risk.head(10))

print("\nMissing values:")
print(inventory_risk.isna().sum())

```

```

Forecast and inventory merged successfully!
Shape: (200, 8)

```

```

Columns:
['sku_id', 'stock_on_hand', 'reorder_point', 'safety_stock', 'forecast_6_week

```

First 10 rows:

	sku_id	stock_on_hand	reorder_point	safety_stock	forecast_6_week_demand
0	1001	5919	2417	1198	119.802438
1	1002	7783	3071	1526	119.812083
2	1003	7089	2770	1374	119.660684
3	1004	9963	3980	1978	120.019339
4	1005	7182	2808	1392	120.646742
5	1006	8453	3306	1642	119.824481
6	1007	5874	2373	1175	120.017444
7	1008	7055	2781	1379	118.876042
8	1009	4380	1789	884	120.842209
9	1010	10543	4248	2113	119.878503

	avg_weekly_forecast	min_weekly_forecast	max_weekly_forecast
0	19.967073	18.417987	22.860635
1	19.968681	18.268207	22.994554
2	19.943447	18.113649	22.983572
3	20.003223	18.417987	23.024297
4	20.107790	18.587716	22.894750
5	19.970747	18.201344	23.089139
6	20.002907	18.445166	22.934744
7	19.812674	18.084599	22.931453
8	20.140368	18.203883	24.944512
9	19.979751	18.147021	23.003006

Missing values:

sku_id	0
stock_on_hand	0
reorder_point	0
safety_stock	0
forecast_6_week_demand	0
avg_weekly_forecast	0
min_weekly_forecast	0
max_weekly_forecast	0

dtype: int64

```
# Calculate inventory coverage and stock position
```

```
inventory_risk["coverage_weeks"] = (  
    inventory_risk["stock_on_hand"]  
    / inventory_risk["avg_weekly_forecast"]  
)  
  
inventory_risk["stock_after_6_week_demand"] = (  
    inventory_risk["stock_on_hand"]  
    - inventory_risk["forecast_6_week_demand"]  
)  
  
inventory_risk["stock_vs_reorder_point"] = (  
    inventory_risk["stock_on_hand"]  
    - inventory_risk["reorder_point"]  
)  
  
inventory_risk["stock_vs_safety_stock"] = (  
    inventory_risk["stock_on_hand"]  
    - inventory_risk["safety_stock"]  
)
```

```

print("Inventory indicators calculated successfully!")

print("\nFirst 10 SKUs:")
print(
    inventory_risk[
        [
            "sku_id",
            "stock_on_hand",
            "forecast_6_week_demand",
            "avg_weekly_forecast",
            "coverage_weeks",
            "stock_after_6_week_demand",
            "stock_vs_reorder_point",
            "stock_vs_safety_stock"
        ]
    ].head(10)
)

print("\nIndicator summary:")
print(
    inventory_risk[
        [
            "coverage_weeks",
            "stock_after_6_week_demand",
            "stock_vs_reorder_point",
            "stock_vs_safety_stock"
        ]
    ].describe()
)

```

First 10 SKUs:

	sku_id	stock_on_hand	forecast_6_week_demand	avg_weekly_forecast \
0	1001	5919	119.802438	19.967073
1	1002	7783	119.812083	19.968681
2	1003	7089	119.660684	19.943447
3	1004	9963	120.019339	20.003223
4	1005	7182	120.646742	20.107790
5	1006	8453	119.824481	19.970747
6	1007	5874	120.017444	20.002907
7	1008	7055	118.876042	19.812674
8	1009	4380	120.842209	20.140368
9	1010	10543	119.878503	19.979751

	coverage_weeks	stock_after_6_week_demand	stock_vs_reorder_point \
0	296.438042	5799.197562	3502
1	389.760354	7663.187917	4712
2	355.455096	6969.339316	4319
3	498.069732	9842.980661	5983
4	357.174998	7061.353258	4374
5	423.269097	8333.175519	5147
6	293.657311	5753.982556	3501
7	356.085208	6936.123958	4274
8	217.473680	4259.157791	2591
9	527.684265	10423.121497	6295

```

stock_vs_safety_stock
0          4721
1          6257
2          5715
3          7985
4          5790
5          6811
6          4699
7          5676
8          3496
9          8430

```

Indicator summary:

	coverage_weeks	stock_after_6_week_demand	stock_vs_reorder_point \
count	200.000000	200.000000	200.000000
mean	373.522635	7338.301325	4498.905000
std	93.094156	1856.100524	1125.011712
min	214.857559	4149.786135	2589.000000
25%	295.584917	5787.893811	3501.750000
50%	374.225585	7360.564055	4498.000000
75%	437.290015	8550.796642	5266.750000
max	610.954713	12069.295546	7377.000000

```

stock_vs_safety_stock
count      200.000000
mean       5989.360000
std        1489.847083
min        3439.000000
25%        4715.500000
50%        5998.500000
75%        6970.000000
max        9795.000000

```

```

# Check how many SKUs are currently below their reorder point
# and safety stock

```

```

below_reorder = (
    inventory_risk["stock_on_hand"]
    < inventory_risk["reorder_point"]
)

```

```

below_safety = (
    inventory_risk["stock_on_hand"]
    < inventory_risk["safety_stock"]
)

```

```

print("Inventory Risk Check")
print("-----")

```

```

print(
    "SKUs below reorder point:",
    below_reorder.sum()
)

```

```

print(
    "SKUs at/above reorder point:",
    (~below_reorder).sum()
)

```

```

print(
    "SKUs below safety stock:",
    below_safety.sum()
)

print(
    "SKUs at/above safety stock:",
    (~below_safety).sum()
)

print("\nReorder point ratio:")
print(
    (
        inventory_risk["stock_on_hand"]
        / inventory_risk["reorder_point"]
    ).describe()
)

```

Inventory Risk Check

```

-----
SKUs below reorder point: 0
SKUs at/above reorder point: 200
SKUs below safety stock: 0
SKUs at/above safety stock: 200

```

```

Reorder point ratio:
count    200.000000
mean      2.521643
std       0.061919
min       2.383214
25%       2.475172
50%       2.519436
75%       2.557449
max       2.676952
dtype: float64

```

```

inventory_risk = inventory_risk.merge(
    skus[
        [
            "sku_id",
            "sku_name",
            "category",
            "subcategory",
            "brand",
            "cost_price"
        ]
    ],
    on="sku_id",
    how="left"
)

print("SKU information merged successfully!")

print("\nShape:", inventory_risk.shape)

```

```

print("\nColumns:")
print(inventory_risk.columns.tolist())

print("\nMissing values:")
print(
    inventory_risk[
        [
            "sku_name",
            "category",
            "subcategory",
            "brand",
            "cost_price"
        ]
    ].isnull().sum()
)

print("\nFirst 5 SKUs:")
print(
    inventory_risk[
        [
            "sku_id",
            "sku_name",
            "category",
            "cost_price",
            "stock_on_hand",
            "forecast_6_week_demand"
        ]
    ].head()
)

```

SKU information merged successfully!

Shape: (200, 17)

Columns:

['sku_id', 'stock_on_hand', 'reorder_point', 'safety_stock', 'forecast_6_week

Missing values:

```

sku_name      0
category      0
subcategory    0
brand         0
cost_price    0
dtype: int64

```

First 5 SKUs:

	sku_id	sku_name	category	cost_price \
0	1001	Electronics_Chargers_1001	Electronics	103.44
1	1002	Dairy_Cream_1002	Dairy	8.96
2	1003	Household_Cleaning_Supplies_1003	Household	7.47
3	1004	Beverages_Juices_1004	Beverages	17.58
4	1005	Household_Paper_Products_1005	Household	13.68

	stock_on_hand	forecast_6_week_demand
0	5919	119.802438
1	7783	119.812083
2	7089	119.660684

3	9963	120.019339
4	7182	120.646742

```

Inventory needed for the next 6 weeks
Calculating safety stock

risk["target_stock"] = (
    'risk["forecast_6_week_demand"]
    'ry_risk["safety_stock"]

Inventory above target
risk["excess_units"] = (
    'risk["stock_on_hand"]
    'ry_risk["target_stock"]
    :0)

Below target
risk["shortage_units"] = (
    'risk["target_stock"]
    'ry_risk["stock_on_hand"]
    :0)

Cost of excess inventory
risk["excess_value"] = (
    'risk["excess_units"]
    'ry_risk["cost_price"]

Cost of required replenishment
risk["reorder_value"] = (
    'risk["shortage_units"]
    'ry_risk["cost_price"]

Inventory impact calculated successfully!")

Print 10 SKUs:")

'risk[
    'sku_id",
    'sku_name",
    'stock_on_hand",
    'forecast_6_week_demand",
    'safety_stock",
    'target_stock",
    'excess_units",
    'shortage_units",
    'excess_value",
    'reorder_value"

    )

```

```

ary:")

'_risk[

'target_stock",
'excess_units",
'shortage_units",
'excess_value",
'reorder_value"

e()

: counts:")
with excess inventory:", (inventory_risk["excess_units"] > 0).sum(
with shortage:", (inventory_risk["shortage_units"] > 0).sum())

```

1	1002	Dairy_Cream_1002	7783
2	1003	Household_Cleaning_Supplies_1003	7089
3	1004	Beverages_Juices_1004	9963
4	1005	Household_Paper_Products_1005	7182
5	1006	Beverages_Coffee_1006	8453
6	1007	Personal_Care_Shampoo_1007	5874
7	1008	Personal_Care_Shampoo_1008	7055
8	1009	Electronics_Batteries_1009	4380
9	1010	Grocery_Rice_1010	10543

	forecast_6_week_demand	safety_stock	target_stock	excess_units	\
0	119.802438	1198	1317.802438	4601.197562	
1	119.812083	1526	1645.812083	6137.187917	
2	119.660684	1374	1493.660684	5595.339316	
3	120.019339	1978	2098.019339	7864.980661	
4	120.646742	1392	1512.646742	5669.353258	
5	119.824481	1642	1761.824481	6691.175519	
6	120.017444	1175	1295.017444	4578.982556	
7	118.876042	1379	1497.876042	5557.123958	
8	120.842209	884	1004.842209	3375.157791	
9	119.878503	2113	2232.878503	8310.121497	

	shortage_units	excess_value	reorder_value
0	0.0	475947.875851	0.0
1	0.0	54989.203733	0.0
2	0.0	41797.184689	0.0
3	0.0	138266.360021	0.0
4	0.0	77556.752563	0.0
5	0.0	65573.520082	0.0
6	0.0	235680.232139	0.0
7	0.0	176549.828155	0.0
8	0.0	208652.254610	0.0
9	0.0	83516.721041	0.0

Summary:

25%	1321.613126	4595.643811	0.0	63416.059932
50%	1593.321014	5879.948696	0.0	107606.966443
75%	1830.351232	6850.514119	0.0	198625.528735
max	2513.704454	9675.295546	0.0	774707.990431

	reorder_value
count	200.0
mean	0.0
std	0.0
min	0.0
25%	0.0
50%	0.0
75%	0.0
max	0.0

Risk counts:

SKUs with excess inventory: 200

SKUs with shortage: 0

```
# Calculate forecast volatility across the 6 forecast weeks
```

```
forecast_volatility = (
    forecast
    .groupby("sku_id")["forecast_demand"]
    .agg(
        forecast_std="std",
        forecast_mean="mean"
    )
    .reset_index()
)
```

```
# Coefficient of variation (CV)
```

```
forecast_volatility["forecast_cv"] = (
    forecast_volatility["forecast_std"]
    / forecast_volatility["forecast_mean"]
)
```

```
# Merge volatility into inventory risk table
```

```
inventory_risk = inventory_risk.merge(
    forecast_volatility[
        [
            "sku_id",
            "forecast_std",
            "forecast_cv"
        ]
    ],
    on="sku_id",
    how="left"
)
```

```
print("Forecast volatility calculated successfully!")
```

```
print("\nFirst 10 SKUs:")
```

```
print(
    inventory_risk[
        [
            "sku_id",
            "forecast_std",
            "forecast_cv"
        ]
    ]
)
```

```

        sku_name ,
        "forecast_std",
        "forecast_cv",
        "excess_units",
        "excess_value"
    ]
].head(10)
)

print("\nVolatility summary:")
print(
    inventory_risk[
        [
            "forecast_std",
            "forecast_cv"
        ]
    ].describe()
)

print(
    "\nMissing volatility values:",
    inventory_risk[
        ["forecast_std", "forecast_cv"]
    ].isnull().sum().sum()
)

```

Forecast volatility calculated successfully!

First 10 SKUs:

	sku_id	sku_name	forecast_std	forecast_cv	\
0	1001	Electronics_Chargers_1001	2.217270	0.111046	
1	1002	Dairy_Cream_1002	2.309209	0.115642	
2	1003	Household_Cleaning_Supplies_1003	2.328235	0.116742	
3	1004	Beverages_Juices_1004	2.276224	0.113793	
4	1005	Household_Paper_Products_1005	2.125282	0.105694	
5	1006	Beverages_Coffee_1006	2.308711	0.115605	
6	1007	Personal_Care_Shampoo_1007	2.267456	0.113356	
7	1008	Personal_Care_Shampoo_1008	2.119058	0.106955	
8	1009	Electronics_Batteries_1009	2.762957	0.137185	
9	1010	Grocery_Rice_1010	2.311355	0.115685	

	excess_units	excess_value
0	4601.197562	475947.875851
1	6137.187917	54989.203733
2	5595.339316	41797.184689
3	7864.980661	138266.360021
4	5669.353258	77556.752563
5	6691.175519	65573.520082
6	4578.982556	235680.232139
7	5557.123958	176549.828155
8	3375.157791	208652.254610
9	8310.121497	83516.721041

Volatility summary:

	forecast_std	forecast_cv
count	200.000000	200.000000
mean	2.277115	0.113955
std	0.259412	0.012159

min	1.386191	0.070032
25%	2.220870	0.111130
50%	2.302673	0.115128
75%	2.346427	0.117465
max	3.774679	0.183223

Missing volatility values: 0

```
# Use the dataset's 75th percentile forecast CV
# as the volatility threshold

volatility_threshold = inventory_risk["forecast_cv"].quantile(0.75)

# Significant excess means excess inventory is greater
# than the safety stock level

inventory_risk["significant_excess"] = (
    inventory_risk["excess_units"]
    > inventory_risk["safety_stock"]
)

# Final inventory decision
def classify_inventory(row):

    # Insufficient inventory
    if row["stock_on_hand"] < row["target_stock"]:
        return "Reorder Now"

    # Significant excess + relatively stable demand
    elif (
        row["significant_excess"]
        and row["forecast_cv"] <= volatility_threshold
    ):
        return "Markdown/Clear"

    # Significant excess + higher volatility
    elif (
        row["significant_excess"]
        and row["forecast_cv"] > volatility_threshold
    ):
        return "Watch/Volatile"

    # Remaining cases
    else:
        return "Healthy"

inventory_risk["decision"] = inventory_risk.apply(
    classify_inventory,
    axis=1
)

print("Inventory risk classification completed!")

print("\nVolatility threshold:")
```

```

print(round(volatility_threshold, 4))

print("\nDecision counts:")
print(
    inventory_risk["decision"].value_counts()
)

print("\nDecision percentages:")
print(
    (
        inventory_risk["decision"]
        .value_counts(normalize=True)
        * 100
    ).round(2)
)

print("\nSample results:")
print(
    inventory_risk[
        [
            "sku_id",
            "sku_name",
            "stock_on_hand",
            "forecast_6_week_demand",
            "excess_units",
            "forecast_cv",
            "decision"
        ]
    ].head(15)
)

```

Inventory risk classification completed!

Volatility threshold:
0.1175

Decision counts:
decision
Markdown/Clear 150
Watch/Volatile 50
Name: count, dtype: int64

Decision percentages:
decision
Markdown/Clear 75.0
Watch/Volatile 25.0
Name: proportion, dtype: float64

Sample results:

	sku_id	sku_name	stock_on_hand	\
0	1001	Electronics_Chargers_1001	5919	
1	1002	Dairy_Cream_1002	7783	
2	1003	Household_Cleaning_Supplies_1003	7089	
3	1004	Beverages_Juices_1004	9963	
4	1005	Household_Paper_Products_1005	7182	
5	1006	Beverages_Coffee_1006	8453	
6	1007	Personal_Care_Shampoo_1007	5874	

7	1008	Personal Care_Shampoo_1008	7055
8	1009	Electronics_Batteries_1009	4380
9	1010	Grocery_Rice_1010	10543
10	1011	Personal Care_Shampoo_1011	7557
11	1012	Beverages_Juices_1012	9048
12	1013	Grocery_Cereals_1013	10465
13	1014	Dairy_Butter_1014	8329
14	1015	Dairy_Cream_1015	7588

	forecast_6_week_demand	excess_units	forecast_cv	decision
0	119.802438	4601.197562	0.111046	Markdown/Clear
1	119.812083	6137.187917	0.115642	Markdown/Clear
2	119.660684	5595.339316	0.116742	Markdown/Clear
3	120.019339	7864.980661	0.113793	Markdown/Clear
4	120.646742	5669.353258	0.105694	Markdown/Clear
5	119.824481	6691.175519	0.115605	Markdown/Clear
6	120.017444	4578.982556	0.113356	Markdown/Clear
7	118.876042	5557.123958	0.106955	Markdown/Clear
8	120.842209	3375.157791	0.137185	Watch/Volatile
9	119.878503	8310.121497	0.115685	Markdown/Clear
10	120.422378	5955.577622	0.109008	Markdown/Clear
11	120.100624	7135.899376	0.112786	Markdown/Clear
12	118.167311	8343.832689	0.093558	Markdown/Clear
13	119.968259	6535.031741	0.111162	Markdown/Clear
14	120.108444	5996.891556	0.115785	Markdown/Clear

```
# Calculate excess inventory percentage
inventory_risk["excess_percentage"] = (
    inventory_risk["excess_units"]
    / inventory_risk["stock_on_hand"]
) * 100

# Normalize excess value and volatility
# so they can contribute to one priority score

excess_value_score = (
    inventory_risk["excess_value"]
    / inventory_risk["excess_value"].max()
)

volatility_score = (
    inventory_risk["forecast_cv"]
    / inventory_risk["forecast_cv"].max()
)

excess_percentage_score = (
    inventory_risk["excess_percentage"]
    / inventory_risk["excess_percentage"].max()
)

# Priority score
# Higher score = more attention required

inventory_risk["priority_score"] = (
    0.5 * excess_value_score
    + 0.3 * excess_percentage_score
    + 0.2 * volatility_score
)
```

```

)

# Rank SKUs
inventory_risk["priority_rank"] = (
    inventory_risk["priority_score"]
    .rank(
        ascending=False,
        method="first"
    )
    .astype(int)
)

print("Inventory priority score calculated successfully!")

print("\nTop 20 priority SKUs:")

top_priority = (
    inventory_risk
    .sort_values("priority_score", ascending=False)
    .head(20)
)

print(
    top_priority[
        [
            "priority_rank",
            "sku_id",
            "sku_name",
            "category",
            "stock_on_hand",
            "forecast_6_week_demand",
            "excess_units",
            "excess_value",
            "forecast_cv",
            "decision",
            "priority_score"
        ]
    ].to_string(index=False)
)

```

demand	excess_units	excess_value	forecast_cv	decision	priority_score
66801	4128.033199	774707.990431	0.118601	Watch/Volatile	0.919860
88146	4043.611854	608604.020125	0.113547	Markdown/Clear	0.805879
64877	3842.335123	585226.062577	0.116912	Markdown/Clear	0.795677
13987	3961.486013	560233.351941	0.119177	Watch/Volatile	0.782214
63073	4316.636927	545191.243824	0.111788	Markdown/Clear	0.769169
18002	3817.281998	532816.221311	0.116778	Markdown/Clear	0.763056
62093	3803.737907	464474.435839	0.154707	Watch/Volatile	0.761289
114307	4101.585693	478737.082143	0.120306	Watch/Volatile	0.733642
02438	4601.197562	475947.875851	0.111046	Markdown/Clear	0.718993
74942	4302.925058	454517.973883	0.108246	Markdown/Clear	0.704770
35032	3779.164968	428141.599278	0.122105	Watch/Volatile	0.700962
62586	3496.437414	432369.450625	0.116514	Markdown/Clear	0.697667

67851	4051.032149	449056.913733	0.104611	Markdown/Clear	0.696706
77298	4195.322702	428426.354284	0.111761	Markdown/Clear	0.692915
95367	4166.704633	412837.095075	0.114399	Markdown/Clear	0.686439
75407	6209.724593	314895.134131	0.162623	Watch/Volatile	0.674891
38655	4159.861345	376467.451685	0.108605	Markdown/Clear	0.653501
25385	9526.474615	356957.003817	0.111832	Markdown/Clear	0.650768
81167	3660.718833	402056.749473	0.085423	Markdown/Clear	0.642483
27339	5935.372661	324783.592027	0.119214	Watch/Volatile	0.633474

```
# Overall inventory impact

total_stock = inventory_risk["stock_on_hand"].sum()
total_forecast = inventory_risk["forecast_6_week_demand"].sum()
total_excess_units = inventory_risk["excess_units"].sum()
total_excess_value = inventory_risk["excess_value"].sum()

print("OVERALL INVENTORY IMPACT")
print("-----")

print(f"Total current stock: {total_stock:,.0f} units")
print(f"Total 6-week forecast demand: {total_forecast:,.0f} units")
print(f"Total excess inventory: {total_excess_units:,.0f} units")
print(f"Total excess inventory value: ₹{total_excess_value:,.2f}")
print(f"Total excess inventory value: ₹{total_excess_value/10_000:.2f} crore")

print("\nDecision Summary:")
print(
    inventory_risk["decision"]
    .value_counts()
)

print("\nAverage inventory coverage:")
print(
    f"{inventory_risk['coverage_weeks'].mean():.1f} weeks"
)

print("\nMedian inventory coverage:")
print(
    f"{inventory_risk['coverage_weeks'].median():.1f} weeks"
)
```

```
OVERALL INVENTORY IMPACT
-----
Total current stock: 1,491,623 units
Total 6-week forecast demand: 23,963 units
Total excess inventory: 1,173,909 units
Total excess inventory value: ₹30,523,695.15
Total excess inventory value: ₹3.05 crore

Decision Summary:
decision
Markdown/Clear    150
Watch/Volatile     50
Name: count, dtype: int64

Average inventory coverage:
```

373.5 weeks

Median inventory coverage:
374.2 weeks

```
# Category-level inventory impact

category_impact = (
    inventory_risk
    .groupby("category", as_index=False)
    .agg(
        sku_count=("sku_id", "count"),
        current_stock=("stock_on_hand", "sum"),
        forecast_6_week_demand=("forecast_6_week_demand", "sum"),
        excess_units=("excess_units", "sum"),
        excess_value=("excess_value", "sum")
    )
)

# Percentage contribution to total excess value
category_impact["excess_value_pct"] = (
    category_impact["excess_value"]
    / category_impact["excess_value"].sum()
) * 100

# Sort by highest excess value
category_impact = category_impact.sort_values(
    "excess_value",
    ascending=False
)

print("CATEGORY-LEVEL INVENTORY IMPACT")
print("-----")

print(
    category_impact.to_string(index=False)
)

print("\nTotal category excess value:")
print(
    f"₹{category_impact['excess_value'].sum():.2f}"
)
```

CATEGORY-LEVEL INVENTORY IMPACT

```
-----
   category  sku_count  current_stock  forecast_6_week_demand  excess_units
Electronics      43      213593      5148.199319  166643.800681
Personal Care     22      153981      2634.307756  121006.692244
    Grocery      25      262442      2991.467930  207994.532070
Household        30      193201      3599.281851  151336.718149
    Snacks       39      331035      4670.609483  260910.390517
    Dairy       25      193173      2997.408622  152306.591378
Beverages        16      144198      1921.459968  113710.540032
```

Total category excess value:
₹30,523,695.15

```
# Select important columns for the final inventory intelligence d

final_inventory_columns = [
    "sku_id",
    "sku_name",
    "category",
    "subcategory",
    "brand",
    "cost_price",
    "stock_on_hand",
    "reorder_point",
    "safety_stock",
    "forecast_6_week_demand",
    "avg_weekly_forecast",
    "min_weekly_forecast",
    "max_weekly_forecast",
    "coverage_weeks",
    "stock_after_6_week_demand",
    "stock_vs_reorder_point",
    "stock_vs_safety_stock",
    "target_stock",
    "excess_units",
    "shortage_units",
    "excess_value",
    "reorder_value",
    "forecast_std",
    "forecast_cv",
    "excess_percentage",
    "decision",
    "priority_score",
    "priority_rank"
]

final_inventory_risk = inventory_risk[
    final_inventory_columns
].copy()

# Save the final dataset
final_inventory_risk.to_csv(
    "final_inventory_risk.csv",
    index=False
)

print("Final inventory risk dataset saved successfully!")

print("\nShape:", final_inventory_risk.shape)

print("\nColumns:", len(final_inventory_risk.columns))

print("\nMissing values:")
print(final_inventory_risk.isnull().sum().sum())
```

```
print("\nSaved file:")  
print("final_inventory_risk.csv")
```

Final inventory risk dataset saved successfully!

Shape: (200, 28)

Columns: 28

Missing values:
0

Saved file:
final_inventory_risk.csv

Start coding or [generate](#) with AI.